

CSS 선택자와 명시도

프론트엔드 기초: 구조와 스타일의 연결

| 학습 체크리스트

- ☐ 태그, 클래스, 아이디 선택자의 구문과 각각의 역할 차이를 설명할 수 있나요?
- ☐ 현대 웹 개발에서 클래스 선택자가 주류가 된 이유와 프레임워크 트렌드를 파악했나요?
- ☐ 클래스와 아이디 선택자가 향후 JavaScript DOM 조작과 어떻게 연계되는지 이해했나요?
- ☐ 명시도(우선순위 점수)의 계산 규칙을 알고 충돌 시 어떤 스타일이 적용될지 예측할 수 있나요?
- ☐ !important 속성의 역할과 남용 시 발생하는 위험성을 인지했나요?

선택자 (Selector) 개요

개념 정의

HTML 문서에서 스타일을 적용할 대상을 **정밀하게 지정**하는 규칙.

1. 태그 선택자 (Type Selector)

- ▶ **정의:** HTML 태그 이름을 기준으로 대상을 직접 지정
- ▶ **구문:** 태그명 { ... }
- ▶ **용도:** 글꼴, 줄간격, 여백 등 사이트 전체의 **기본 스타일 초기화** 시 적합

```
/* 페이지 내 모든 p 태그에 일괄 적용 */  
p { color: blue; }
```

클래스와 아이디 선택자

2. 클래스 선택자 (Class Selector)

- ▶ **정의:** 요소의 class 속성값을 기준
- ▶ **구문:** .클래스명 { ... }
- ▶ **특징:** 여러 요소에 중복 부여 가능
- ▶ **용도:** 동일한 디자인의 **재사용성 극대화**

```
/* 해당 클래스명이 부여된 모든 요소 */  
.error-text {  
  color: red;  
}
```

3. 아이디 선택자 (ID Selector)

- ▶ **정의:** 요소의 고유한 id 속성값을 기준
- ▶ **구문:** #아이디명 { ... }
- ▶ **특징:** 페이지 내 단 하나만 존재해야 함
- ▶ **용도:** 독자적인 고유 스타일 부여 (우선순위 높음)

```
/* 메인 헤더 영역 등 고유 요소 */  
#main-header {  
  background-color: darkblue;  
}
```

실무 개발 트렌드와 팁

📦 클래스 중심 개발

- ▶ **과거:** 구조적이고 복잡한 설계
div > ul.menu > li a
- ▶ **현재:** 유지보수성을 위한
클래스 중심 네이밍
- ▶ **트렌드:** Bootstrap, TailwindCSS 등 유틸리티 프레임워크 대세

📄 JavaScript 연계

- ▶ **개념:** 화면을 동적 제어하는 **DOM (Document Object Model)**과 직결
- ▶ **연계:** CSS 선택자 규칙을 JS에서 100% 동일하게 재사용
- ▶ `document.querySelector()`
`document.getElementById()`

🔗 학습 도구 추천

- ▶ **CSS Diner**
flukeout.github.io
- ▶ **특징:** 32단계 미니 게임을 통해 대표적인 유형의 선택자를 인터랙티브하게 학습 가능

명시도 (Specificity) 계산 규칙

여러 CSS 규칙이 충돌할 때, 우선하여 적용할 규칙을 가려내는 **우선순위 점수 체계**. 가장 높은 점수를 획득한 스타일 선언이 반영됨.

CSS 선언 방식	계산 점수	설명
!important	무한대	모든 명시도 점수 규칙을 무력화하고 최우선 적용
인라인 스타일 (Inline)	1000점	HTML 태그 내 style="..." 속성에 직접 선언
아이디 선택자 (ID)	100점	#id-name
클래스/가상클래스 (Class)	10점	.class-name, :hover 등
태그 선택자 (Tag)	1점	p, h1, div 등

- **동점 시 우선순위 (Cascading 규칙):** 명시도 점수가 완전히 같을 때는 **가장 아래(나중)에 작성된 스타일**이 덮어씀

!important 의 개념과 위험성

극단적 우선순위 선언

스타일 속성값 뒤에 붙어 해당 선언의 명시도 점수를 무한대로 끌어올리는 속성입니다.

```
p {  
  color: red !important; /* 모든 인라인, ID, 클래스  
  설정을 덮어쓰고 무조건 빨간색으로 고정 */  
}
```

⚠ 실무적 위험성

▶ 상속 파괴

자연스러운 CSS 상속 및 덮어쓰기 흐름 (Cascading)을 단절시킵니다.

▶ 스파게티 코드 유발

다른 디자인 수정 시 더 강력한 !important를 남발해야 하는 악순환을 초래합니다.

▶ 실무 대원칙

서드파티 라이브러리의 강제 오버라이딩 등 극히 이례적인 상황을 제외하면 **절대 사용 금지**를 권장합니다.